



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 07

NOMBRE COMPLETO: Bueno Hernández Héctor Daniel

N° de Cuenta: 319233148

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 05/04/2025

CALIFICACIÓN: _____

Agregar movimiento al helicóptero

Se solicitó agregar movimiento hacia adelante y hacia atrás al modelo del helicóptero ya presente en la escena. Para ello, se reutilizó el código generado para el movimiento del carro, pero cambiando los nombres de las variables con el propósito de controlar el movimiento de cada modelo con distintas teclas.

Se declaró la variable `gettransx_helicopter` dentro del archivo *Windows.h* y esta se utiliza dentro de *Windows.cpp*. Dentro de este, se generó un fragmento de código en el que, al presionar la tecla **X** la variable `gettransx_helicopter` aumenta en 1 su valor, y al presionar la tecla **C** la variable `gettransx_helicopter` disminuye en 1 su valor.

```
if (key == GLFW_KEY_X)
{
    theWindow->transx_helicopter += 1.0; //avanzar sobre el eje x 5 unidades
}
if (key == GLFW_KEY_C)
{
    theWindow->transx_helicopter -= 1.0; //retroceder sobre el eje x 5 unidades
}
```

Ya en el `main`, lo única modificación que se hizo al instanciar el modelo del helicóptero fue cambiar la línea correspondiente a la transformación `translate`. En esta, a la coordenada en el eje X, se le suma el valor de la variable `gettransx_helicopter`. De tal modo, al presionar las teclas antes mencionadas, la coordenada en X del modelo aumenta o disminuye respectivamente.

```
//helicoptero
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f + mainWindow.gettransx_helicopter(), 5.0f, 6.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Blackhawk_M.RenderModel();
```

Luz spotlight del helicóptero

Posteriormente, se solicitó agregar una luz de tipo *spotlight* al helicóptero, esta debe ser de color amarillo y debe apuntar hacia el suelo. Para esto, la luz se declaró de la misma forma que se hizo en el ejercicio en clase.

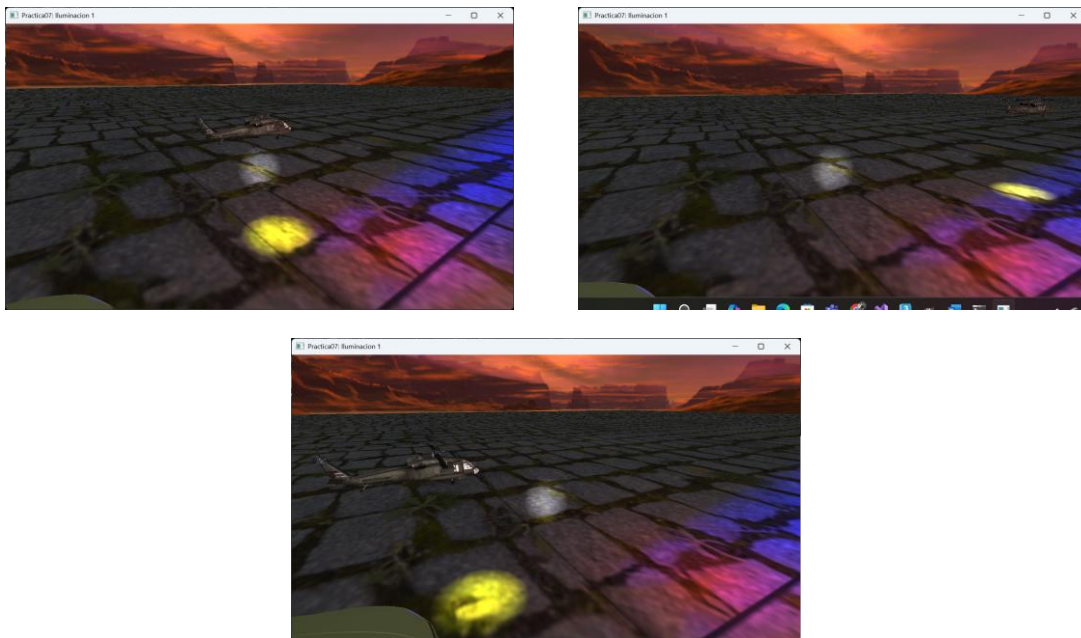
Se declaró un tercer objeto de la clase `SpotLight` dentro del arreglo `spotLights[]`. Este objeto es de color amarillo y se encuentra situado en el origen. Asimismo, su dirección también se declara en 0 en todos los ejes. Finalmente, se especifica un ángulo de amplitud del cono de la luz de 20.

```
spotLights[3] = SpotLight(1.0f, 1.0f, 0.0f,
    0.5f, 1.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    20.0f);
spotLightCount++;
```

Después, al igual que los faros del carro, se usa la función `SetFlash` para poder cambiar las propiedades de la luz en tiempo de ejecución, en este caso, la posición. La función recibe como argumentos la posición de la luz y su dirección. La posición utilizada contiene nuevamente el valor de `gettransx_helicopter` para que esta cambie al presionar las teclas y las coordenadas son las mismas que las del helicóptero en los ejes Y y Z, pero en X se encuentra un poco por delante del centro del helicóptero para que esta simule que sale del frente del helicóptero.

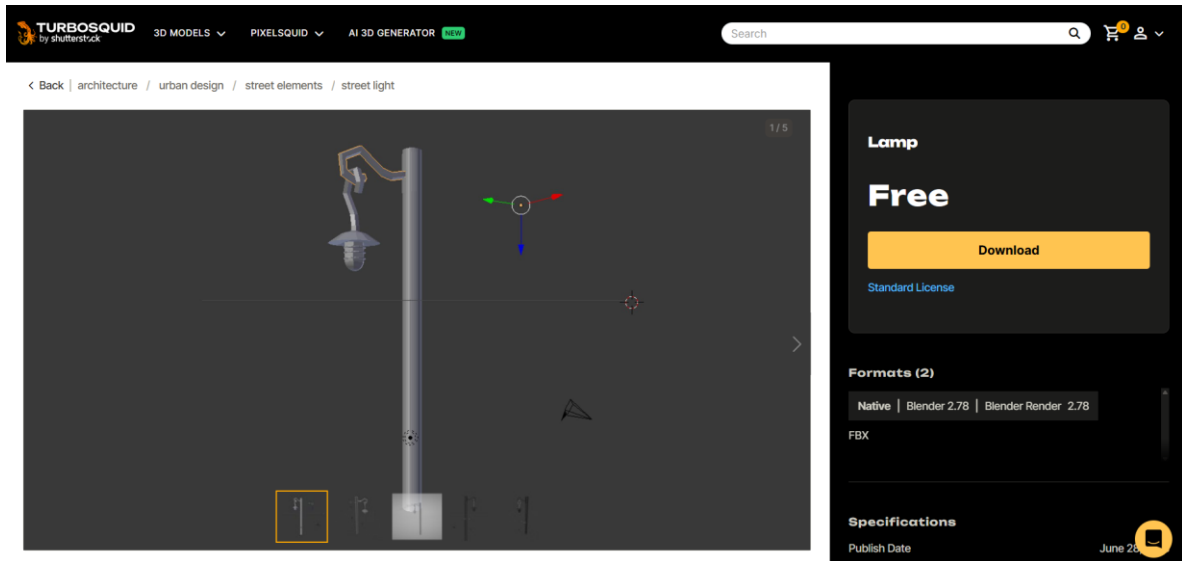
```
spotLights[0].SetFlash(lowerLight, camera.getCameraDirection());
spotLights[1].SetFlash(glm::vec3(10.0f + mainWindow.gettransx_carro(), 0.8f, 0.0f), glm::vec3(-1.0f, 0.0f, 0.0f));
spotLights[2].SetFlash(glm::vec3(10.0f + mainWindow.gettransx_carro(), 0.8f, 4.0f), glm::vec3(-1.0f, 0.0f, 0.0f));
spotLights[3].SetFlash(glm::vec3(-1.0f + mainWindow.gettransx_helicopter(), 5.0f, 6.0f), glm::vec3(0.0f, -1.0f, 0.0f));
```

La ejecución del código se ve de la siguiente forma:



Poste de luz con luz puntual

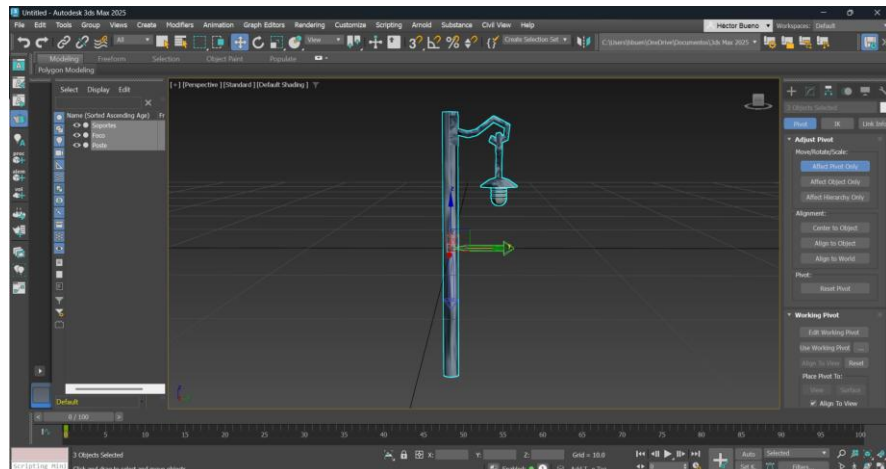
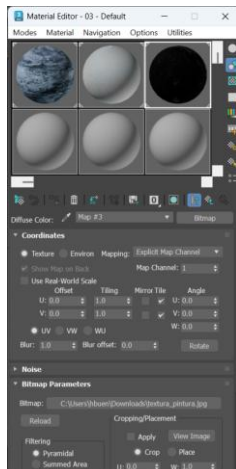
Se solicitó agregar a la escena el modelo de una lámpara con texturas, la cual debe contar con una luz puntual de color blanco. Para ello, primero se buscó un modelo 3D de una lámpara. En mi caso, debido que el personaje que usaré en el proyecto es Link de The Legend Of Zelda: Wind Waker, estaba buscando un modelo que semejara el estilo artístico del juego. Al final, el modelo seleccionado fue el siguiente:



Con respecto a las texturas, busqué una textura distinta para el poste, el foco y para los toroides que se encuentran sujetando el foco. Las imágenes usadas para las texturas fueron las siguientes:



Dentro de 3ds Max, el modelo se escaló a un tamaño adecuado y se crearon los materiales para agregar las texturas antes mencionadas. Adicionalmente, se modificó la posición del pivote para que este se encontrara en el centro del poste de luz y se colocó en el origen. Finalmente, se exportó el modelo en formato *.obj*.



En el código, se declaró un objeto de la clase `Model` llamado `Poste_M` y este se inicializó con la dirección de la ruta en la que se encuentra el archivo del modelo.

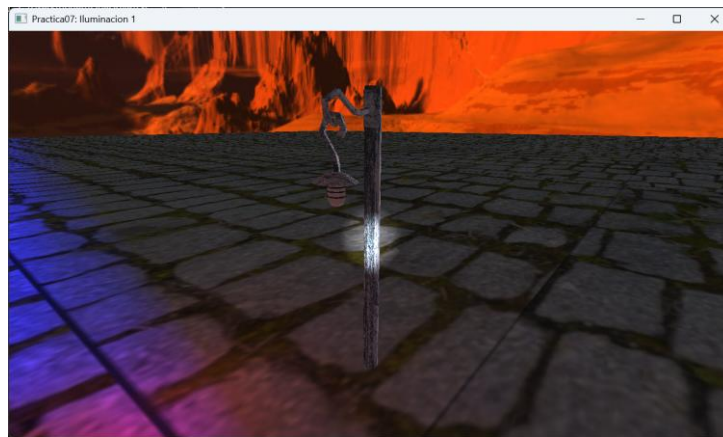
```
Model Kitt_M;
Model Llanta_M;
Model Elantra_M;
Model Cofre_M;
Model Blackhawk_M;
Model Poste_M;

Cofre_M = Model();
Cofre_M.LoadModel("Models/Coche/cofre.obj");
Blackhawk_M = Model();
Blackhawk_M.LoadModel("Models/uh60.obj");
Poste_M = Model();
Poste_M.LoadModel("Models/poste_luz.obj");
```

Después, dentro del `main` se declaró de la misma forma que los demás modelos presentes en el escenario. Se declaró una nueva matriz y se le aplicaron las transformaciones necesarias para que este se pudiera visualizar adecuadamente y en su posición correspondiente para finalmente mandar a llamar la función que se encarga de renderizar el modelo.

```
//poste de luz
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-3.0f, 3.7f, -6.0));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Poste_M.RenderModel();
```

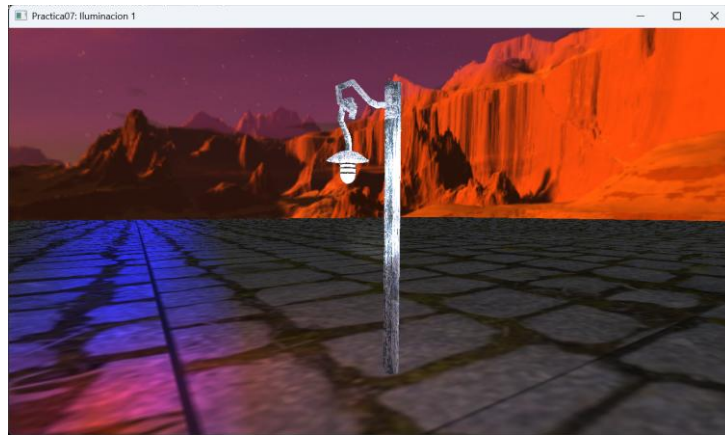
Ya en la ejecución, el modelo de la lámpara se ve de la siguiente forma:



Con respecto a la luz, se usó como referencia la luz puntual ya declarada en el código proporcionado por el profesor. Se agregó un nuevo objeto de la clase `PointLight` dentro del arreglo `pointLights[]`. Este se declaró de color blanco, con una *radiación* de 1.0 y una *intensidad* de 0.1. Además, esta se encuentra colocada en las coordenadas, $(-3.0, 5.7, -5.5)$, las cuales corresponden a la posición del foco de la lámpara.

```
//luz del poste
pointLights[1] = PointLight(1.0f, 1.0f, 1.0f,
    1.0f, 0.1f,
    -3.0f, 5.7f, -5.5,
    0.3f, 0.2f, 0.1f);
pointLightCount++;
```


Al ejecutar el código, el modelo de la lámpara ya con la luz creada se ve de la siguiente manera:



Problemas presentados

No presenté problema alguno en la realización de esta práctica debido a que las actividades solicitadas se podían realizar de la misma forma que se hizo el ejercicio en clase o, en el caso de la luz puntual, solamente fue necesario observar el código de la luz ya declarada para poder declarar una nueva luz con las propiedades deseadas. Adicionalmente, no conté con problema alguno con respecto a la escritura del código ni con la ejecución del mismo.

Conclusión

Considero que la dificultad de esta práctica fue bastante adecuada tomando en cuenta que se trata de una práctica introductoria al tema de iluminación. Los ejercicios solicitados me permitieron trabajar con distintos tipos de luces y las tareas requerían que analizara el código proporcionado para poder saber donde realizar las modificaciones correspondientes. Además. Estos ejercicios engloban los conocimientos de múltiples prácticas anteriores como el texturizado y el manejo de modelos en 3ds Max y en OpenGL.

Creo que la explicación del profesor fue bastante buena ya que se tomó el tiempo de explicar detalladamente los distintos tipos de luces existentes, las diferencias entre ellas y los fragmentos de código correspondientes. Durante la sesión estuvimos realizando modificaciones a las propiedades de las luces para ver la diferencia en ejecución, lo que me permitió poder obtener las luces con las características deseadas de forma más rápida al momento de hacer la práctica.

Bibliografía

- rawpixel.com. (s. f.). *Redirect notice*. Freepik.
<https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.freepik.com%2Fphotos%2Fblack-paint-texture&psig=AOvVaw0YloWOejJGsWBdOqA7U9gZ&ust=1743920820718000&source=images&cd=vfe&opi=89978449&ved=0CBcQjhqxqFwoTCNjP0-e4wlwDFQAAAAAdAAAAABAE>
- Shirey, T., & Shirey, T. (s. f.). *Explore this Grungy Lamp Post: Texture Pack*. WebFX.
<https://www.webfx.com/blog/web-design/grungy-lamp-post-texture-pack/>
- Shutterstock. (s. f.). *Free Lamp Model*. Turbosquid. Recuperado 4 de abril de 2025, de <https://www.turbosquid.com/3d-models/lamp-model-1300754>
- Wanchanta. (2017, 14 junio). *Redirect notice*. iStock.
<https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.istockphoto.com%2Fes%2Ffotos%2Fglass-texture&psig=AOvVaw3xnn0fGe2b-BmDKABrcD2X&ust=1743920857075000&source=images&cd=vfe&opi=89978449&ved=0CBcQjhqxqFwoTCODAkcuhwIwDFQAAAAAdAAAAABAJ>